

PART I: EXECUTIVE REPORT

Subject: Accelerating and Scaling our UVC Disinfection Perception Pipeline using NVIDIA Technologies

Current State:

Our current perception pipeline successfully identifies disinfection hotspots (door handles, bed rails) by taking offline data (Video, ROS Bags, RViz images) and processing them through cloud APIs (Gemini) and heavy AI models (SAM 3D). While effective for prototyping, this "batch/offline" approach faces three future roadblocks:

1. **Hospital Data Privacy (HIPAA):** Sending patient room video to cloud APIs (Gemini) is a major compliance risk.
2. **Latency:** Cloud processing prevents the robot from adapting in real-time.
3. **Data Scarcity:** Manually driving the robot into new rooms to record videos and point clouds to test our software does not scale.

These 3 states are currently our risk states, so we need to fix these first using NVIDIA ...kesav

The NVIDIA Solution:

NVIDIA provides a complete hardware and software stack to move our brain from the *Cloud* to the *Robot*, and to move our testing from *Physical Rooms* to *Virtual Simulation*.

Key Recommendations & ROI:

1. **Adopt NVIDIA Jetson Orin (Edge Hardware):** By putting an NVIDIA Jetson AGX Orin computer on our physical UVC robot, we can run Vision-Language Models (replacing Gemini) gemini is better kesav, but still vlm is more effective for running images and text and we get a output locally, and SAM 3D *locally*. **ROI:** Zero cloud latency, zero ongoing cloud API costs, and total data privacy (no video leaves the hospital).
2. **Utilize NVIDIA Isaac Sim (Training/Simulation):** Instead of recording physical hospital rooms, we can generate thousands of photorealistic virtual hospital rooms. **ROI:** We can automatically train our models on perfectly labeled 3D hotspots without human manual annotation. {We can use NVIDIA Isaac Sim to create realistic virtual hospital rooms and automatically generate perfectly labeled hotspot data, so we don't need to manually record or annotate real environments, saving a lot of time and cost while improving our model training.}
3. **Leverage NVIDIA Isaac ROS:** NVIDIA offers hardware-accelerated ROS packages. We can use these to fuse our camera and point-cloud data instantly, eliminating the need to process clumsy "RViz screen grabs."

Conclusion : Transitioning our current Python pipeline to the NVIDIA ecosystem will transform our robot from an "offline data processor" into an "autonomous, real-time, privacy-compliant medical device."

PART II: NVIDIA & ROBOTICS

1. What is NVIDIA in the context of Robotics?

NVIDIA is no longer just a graphics card company; they are the backbone of modern AI compute. In robotics, NVIDIA provides a unified architecture based on **CUDA** (a parallel computing platform). This means code you write to process 3D point clouds on a massive cloud server can scale down and run on the small computer inside your robot.

NVIDIA's robotics ecosystem is divided into two :

- **The Brain on the Robot (Edge AI):** NVIDIA Jetson hardware and Isaac ROS software.
- **The Brain in the Lab (Training & Sim):** NVIDIA Omniverse and Isaac Sim.2. NVIDIA Robotics Training Platforms (Omniverse & Isaac)

2. NVIDIA Omniverse

Omniverse is a platform for building and operating custom 3D pipelines. Think of it as the "operating system for 3D worlds."

- **How it applies to you:** Hospital environments are complex. Omniverse allows you to import CAD models of hospital beds, IV poles, and doors to create a "Digital Twin" of a hospital ward.

NVIDIA Isaac Sim (The Crown Jewel for our project)

Built on top of Omniverse, Isaac Sim is a photorealistic, physically accurate robotics simulator.

- **Synthetic Data Generation (SDG):** This is massive for our project. Right now, we rely on physical ROS bags and AVIs. With Isaac Sim's "Replicator", we can generate 100,000 images of hospital rooms with varying lighting, different IV pole positions, and *perfectly generated 2D bounding boxes and 3D point clouds*.
- **Reinforcement Learning (Isaac Lab):** If you ever want the robot arm/base to learn how to dynamically angle the UVC light based on the hotspot pose, it can learn in simulation millions of times faster than real life.

The key solution for our project

[We can use NVIDIA Omniverse to build a digital twin of hospital environments by importing real equipment models like beds, IV poles, and doors, which helps us simulate realistic hospital setups.

On top of that, NVIDIA Isaac Sim allows us to generate large-scale synthetic data, like thousands of hospital room images with different lighting and object positions, all with automatic and accurate annotations, so we can train our models without relying only on real-world recordings.

Additionally, using Isaac Sim's reinforcement learning capabilities, we can train the robot to intelligently adjust things like UVC light angles based on hotspot positions in simulation, which is much faster and more scalable than doing it in the real world.]3. Cloud vs. Edge in Robotics (and the Concept of "Parsing")

3. What is Cloud in Robotics?

The "Cloud" means processing data on remote servers (AWS, Google Cloud, NVIDIA DGX Cloud) rather than on the robot itself.

- **What Cloud SHOULD be used for:** Fleet management (tracking where our UVC robots are), heavy training of new AI models, storing historical ROS bags, and generating simulated environments.
- **What Cloud SHOULD NOT be used for (in your project):** Real-time hotspot detection. Sending live video and PointCloud2 data over hospital Wi-Fi to a cloud API (like Gemini) is slow, drops packets, and violates strict medical data privacy laws.

What is "Parsing" in our context?

Parsing means extracting meaningful structure from raw data.

- **What to Parse in the Cloud:** Offline batch records, JSON files, historical priority lists.
- **What NOT to Parse in the Cloud: RViz Screen grabs.** In our current doc, you mention using RViz screen grabs to help triangulate 3D points. RViz is a *visualization tool for humans*, not a data source for machines. Sending pictures of RViz to the cloud to run AI on them is highly inefficient. On NVIDIA hardware, we would parse the raw math (Camera Intrinsic matrix + PointCloud coordinates) directly using CUDA, bypassing the need for "screen grabs" entirely.

In robotics, the cloud refers to using remote servers for processing instead of doing everything on the robot itself. It's very useful for things like fleet management, storing data, training AI models, and running large simulations. However, for our use case, we should avoid using the cloud for real-time hotspot detection, because sending live video and sensor data over hospital networks can be slow, unreliable, and may also create data privacy concerns.

Another important concept is "parsing," which simply means extracting useful information from raw data. In our case, we should handle critical, real-time data processing directly on the robot using efficient methods instead of sending visual outputs like RViz screen captures to the cloud. Tools like RViz are meant for human visualization, not for machine processing, so it's much more efficient to work directly with the raw sensor data and computations on-device.

4. NVIDIA Jetson Training Platforms (The Edge AI)

What are they?

NVIDIA Jetson is a family of small, low-power computers designed to go *inside* the robot, but they possess massive GPU cores capable of running AI models locally.

The Current Lineup:

1. **Jetson Nano / TX2:** Older, very weak. (Do not use).
2. **Jetson Orin Nano / NX:** Mid-range. Good for basic cameras and navigation.
3. **Jetson AGX Orin:** The powerhouse. **This is what our UVC robot needs.** It has up to 275 TOPS (Trillion Operations Per Second) of AI compute.

How is it used?

we install **JetPack SDK** (NVIDIA's Linux OS with all AI drivers). we then take our SAM 3D models and our VLMs (Vision Language Models) and optimize them using **NVIDIA TensorRT**. TensorRT shrinks the AI models so they run blazingly fast on the Jetson.

[We can use the NVIDIA Jetson AGX Orin as the main computing unit on our UVC robot, since it provides very high AI performance for real-time processing. We install the JetPack SDK, which is NVIDIA's optimized operating system with all required drivers, and then deploy our AI models like SAM 3D and Vision Language Models.

These models are further optimized using NVIDIA TensorRT, which reduces their size and makes them run much faster and more efficiently directly on the robot. NVIDIA TensorRT is a high-performance deep learning inference SDK created by NVIDIA. It optimizes trained neural networks for deployment on NVIDIA GPUs, delivering low latency and high throughput across data center, edge, cloud, and consumer devices. TensorRT plays a central role in NVIDIA's AI inference platform, complementing training frameworks such as TensorFlow, PyTorch, and MXNet.]

5. Robot Development: What is Possible what is Not Possible?

What IS Possible:

- **Running Local LLMs/VLMs:** we can replace the Google Gemini API with an open-source VLM (like LLaVA or NVIDIA's VILA) running entirely offline on the Jetson AGX Orin.
- **Real-time Point Cloud + Video Fusion:** Using NVIDIA Isaac ROS nvblox or depth_image_proc running on CUDA, we can fuse the RGB video and PointCloud2 at 30+ frames per second.
- **Sensor Fusion without RViz:** we can perform mathematical projection of 2D bounding boxes into 3D metric space instantly, making the "RViz screen grab" triangulation step obsolete.

What is NOT Possible (Limitations):

- **Infinite battery life:** Jetson AGX Orin pulls up to 60W of power. our robot's battery management system must account for this. like typically low keshav

zero-shot perfection: Even with NVIDIA, off-the-shelf zero-shot models will occasionally misidentify a complex medical device. We still need a robust fallback fusion module (which we have brilliantly included in our fusion_module.py).

[We integrate this optimized pipeline directly into our fusion module, ensuring efficient real-time processing on the robot]

[fusion_module.py is the part of our code where all sensor data is merged to make smarter decisions] it combines inputs from multiple sources into one meaningful output keshav

PART III: HOW TO RUN OUR *PROJECT* ON NVIDIA

Phase 1: Moving from Offline APIs to Local Edge Inference

Currently, our pipeline uses `auto_annotate_with_gemini.py` and calls Google APIs.

1. **Hardware setup:** Obtain a Jetson AGX Orin Developer Kit.
2. **Software swap:** Use **NVIDIA NIM (NVIDIA Inference Microservices)** to deploy a local Vision Language Model on the Jetson.
3. **Result:** our robot can look at an RGB frame, identify "door_handle" and assign "high priority," *without an internet connection*.

Phase 2: Fixing the "RViz Grabs" Triangulation Problem

we struggle with using RViz screenshots to help map 2D video to 3D point clouds.

- **The NVIDIA Fix:** Stop using RViz images. Use **NVIDIA Isaac ROS**.
- **How:** we feed the RGB video and `sensor_msgs/PointCloud2` directly into Isaac ROS. Using NVIDIA's hardware-accelerated **cuPCL (CUDA Point Cloud Library)**, the system mathematically maps the 2D bounding box from the VLM directly onto the 3D point cloud cluster instantly. It automatically outputs the exact `position_3d [x,y,z]` goal point for the UVC light.

Phase 3: Scaling Training with Isaac Sim

mention having only ~30 manual sessions (triples of bag/video/rviz). This is not enough data for a robust commercial product.

1. **Import:** Buy or download 3D CAD assets of hospital rooms (beds, IV poles, doors). Import them into **Isaac Sim**.
2. **Simulate:** Attach a virtual camera and virtual depth sensor to a virtual robot in Isaac Sim.
3. **Generate:** Tell Isaac Sim to randomly change the lighting, move the beds around, and drive the robot. Have it output exactly the format you need: Video + ROS Bag + perfectly accurate JSON Hotspot lists.
4. **Result:** jump from 30 test environments to 10,000 test environments overnight.

Phase 4: Running Your Enhanced Inference Script

Our current script is:

```
python scripts/enhanced_inference.py --video video.avi --bag recording.bag
```

On NVIDIA Jetson, this evolves to a real-time ROS2 Node:

Instead of a Python script parsing batch files, we wrap our `fusion_module.py` and `enhanced_inference_engine.py` into a ROS2 Node.

As the robot drives, the node subscribes to live `/camera/rgb` and `/camera/depth/points`. The Jetson's

GPU runs the VLM and SAM 3D in parallel, constantly publishing our JSON Hotspot list dynamically to the robot's navigation stack so it can point the UVC light on the fly.

What should we upgrade and improve to achieve our future goals:

Architectural Upgrades (Software & Infrastructure)

Right now, our system processes recorded data offline using scripts, but we should move to a real-time setup using ROS 2, where the robot directly processes live camera and depth data and continuously outputs hotspot information while operating. We also need to remove our current dependency on RViz screen captures, since they are not reliable for machine processing. Instead, we can directly compute accurate 3D positions using proper camera math and transformation frameworks, which will be faster and more scalable.

AI Model Upgrades (Privacy & Latency)

Currently, we rely on cloud APIs like Google Gemini, which introduces latency and raises serious data privacy concerns in hospital environments. We should replace this with local Vision-Language Models running directly on the robot. Additionally, we can significantly improve performance by optimizing our models using NVIDIA TensorRT, making them faster and more efficient for real-time use.

Data & Training Upgrades (Scaling)

Instead of manually collecting limited real-world data, we can use NVIDIA Isaac Sim to generate large-scale synthetic datasets. By creating virtual hospital environments, we can produce thousands of labeled training samples with different variations, which will help us improve model accuracy and reduce dependency on physical testing.

Hardware Upgrades

To make the system fully autonomous, we should move all processing onto the robot itself by integrating the NVIDIA Jetson AGX Orin. This will allow the robot to handle perception, decision-making, and control entirely on-device without relying on external systems.

Executive Summary

Our goal is to transform the current system from an offline, cloud-dependent prototype into a real-time, fully autonomous, and privacy-compliant solution.

We will ensure that all hospital data stays secure by running AI models locally on the robot instead of sending data to the cloud. At the same time, we will upgrade the system to process data in real time, allowing the robot to detect and respond to hotspots dynamically as it moves.

We will also remove inefficient methods like using visualization screenshots and replace them with direct mathematical computations for better accuracy and speed. Finally, by adopting simulation tools, we can scale our testing and training process significantly without relying only on physical environments

future plan:

We need to integrate the Jetson AGX Orin into the robot, transition our software to a ROS 2-based real-time architecture, and replace cloud-based AI services with optimized local models.

First, by adding the NVIDIA Jetson AGX Orin inside the robot, we are basically giving the robot its own powerful brain, so it can process everything on its own instead of depending on an external laptop or server.

Second, moving to ROS 2 means our software will shift from running after recording data to working in real time, so the robot can detect hotspots and respond instantly while moving.

Third, replacing cloud-based AI with local models means we stop sending data to external services and instead run everything directly on the robot, which improves speed, reliability, and ensures sensitive hospital data stays completely private.
